Lucas Swanson
6/3/2026
Software Engineering Methods

Lab 9

Part A:
Track Selection – Frontend
Deployment priorities –

a. Operational simplicity
b. Maintainability

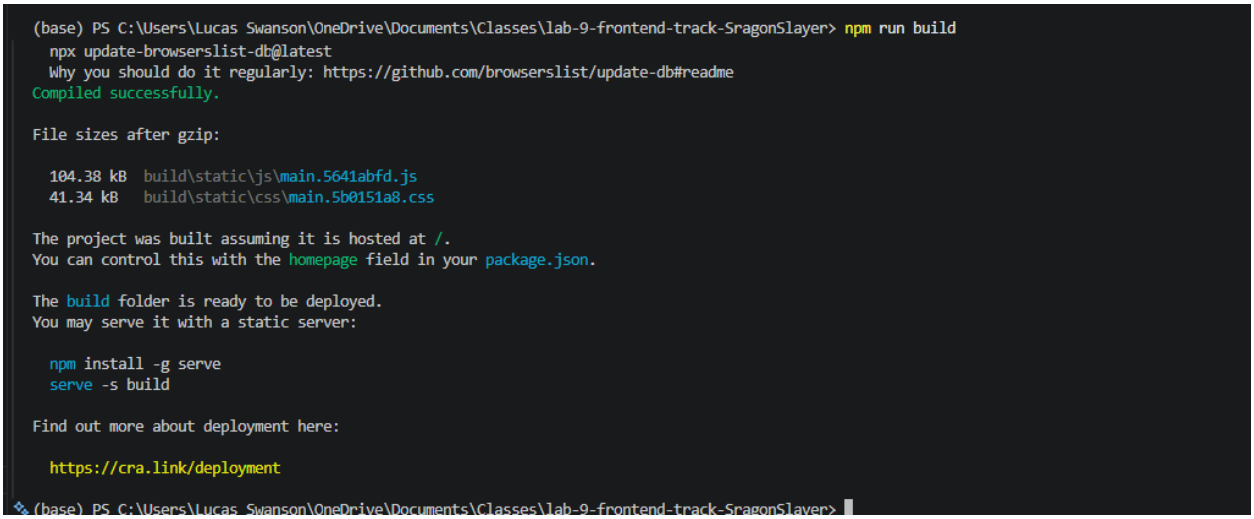
I selected this track because I believe it is the one I would use most often in my day-to-day activities. I also think this would be more beneficial to me in the short term, considering our final project would be somewhat similar.

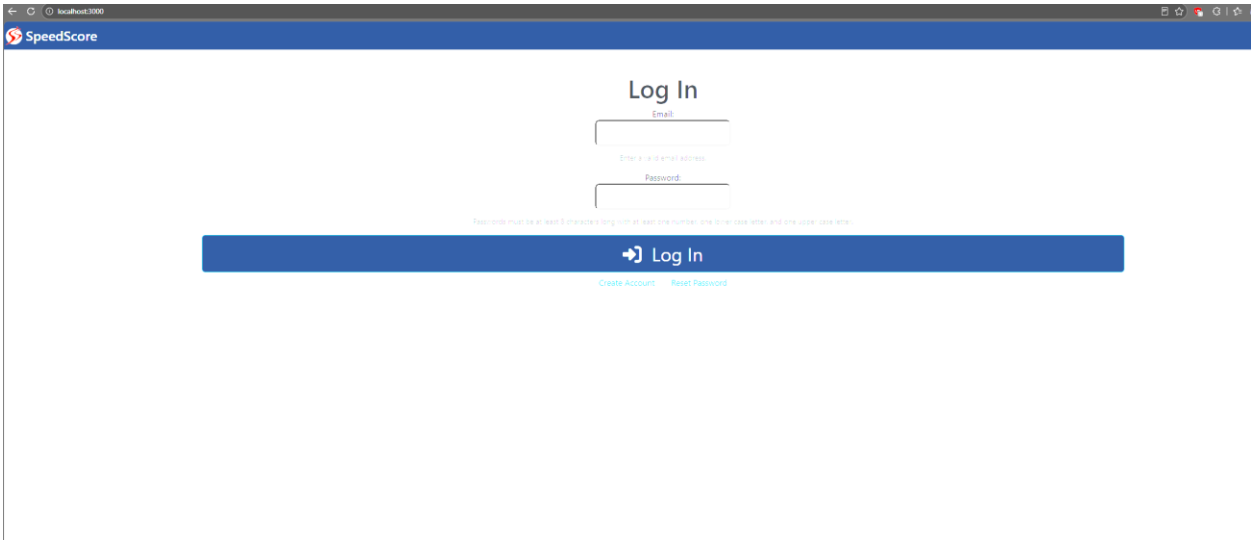
Part B:

Dimension	Virtual Machine	Kubernetes	Serverless
Operational Complexity	High	Medium	Low
Control	High	High	Low
Deployment Automation	Medium	High	High
Reliability	Determined by team skill	High	High
Maintainability	Low	Medium	Low

Here we are going to prioritize the Operational complexity (prioritizing the lowest values) as well as the Maintainability. Based on these Kubernetes seems to be the best option for my deployment priorities due to the highest maintainability, not too high operational complexity, and high amounts of control, automation and reliability.

Part C:





Here we had to upload some dependencies to get this to run, namely the .env file, node_modules for all npm dependencies, and index.ts in order to set up some global variables and other package information. To be honest I tried to deploy this using Amazon AWS services as well as Pulumi.com, but getting these to deploy correctly has been an issue gaining access to the repository and was not feasible. So I opted for a local server host deployment.

Part D:

1. Steps:
    a. Install npm dependencies
    b. Build the containerized package
    c. Run the build on host machine to deploy the server
2. Pain points:

    a. Attempted to deploy the server on multiple Kubernete compatible hosting server methods via AWS as well as Pulumi, and getting the websites access to the repostitory was a big issue.
    b. Getting all deployment configurations is complex and challenging without proper templates.
3. Responses:
    a. Attempted creating a personal repository containing the server file information so I can grant the websites access.
    b. I attempted to use generative AI to assist in creating the configuration files for the server deployment settings, and learned that there is an incredible amount of options when configuring these servers, even if you already have a template in mind. These can be quite in depth and customizable and is difficult to approach as someone who has never worked on a website before.
4. Advice:
    a. Read up on how to host website servers and what companies offer small versions of this for free.
    b. Build your own first! Before you try to apply the code hosted in a school repository, make your own dummy website that is blank first before attempting this as simply navigating the hosting services websites can prove difficult.

Part E:

Process / Verification:

1. Ensure you have Git, Docker Desktop, kubectl, Pulumi CLI, and npm all installed.
2. Clone your repository, then create your docker image “docker build -t (your frontend app name)”.
3. Push the image to a container registry
4. Initialize Pulumi
5. Create Kubernetese resources.
    a. Use the online Pulumi website to define a deployment resource and service resource
6. Deploy the Kuberenetes
    a. Run “pulumi up” and review the output log to confirm
7. If you want this on a CI/CD pipeline, configure a GitHub action to execute this on a push to main

Common issues:

1. Kubernetes must be running before you deploy!!
2. Ensure kubectl is connected to the correct cluster.
3. Docker images must be pushed before Pulumi can deploy them